Үйлдлийн систем — Лабораторийн ажил №4

Сэдэв: Process State, Process Priority, Context Switching-ийн симуляци

1. Зорилго

Энэхүү лабораторийн ажлын зорилго нь үйлдлийн системийн процесс удирдлагын дараах чухал ойлголтуудыг програмчлалын түвшинд **загварчлан** ойлгоход оршино. Үүнд:

- **Process Control Block (PCB)**
- **Process State**
- **Process Priority**
- **CPU Scheduling**
- **Context Switching**

Тус лабораторийн хүрээнд оюутнууд:

- Процесс бүрийн мэдээллийг **PCB бүтэц** ашиглан хадгалж сурна.
- Процессын **READY, RUNNING, WAITING, TERMINATED** зэрэг төлөвүүдийг програмын түвшинд загварчилна.
- Процесс бүрт **priority** оноож, аль процесс түрүүлж CPU авах ёстойг шийдэх логик хэрэгжүүлнэ.
- Нэг процессоос нөгөө процесс руу шилжих үед **context switching**-ийн ойлголтыг симуляци байдлаар хэрэгжүүлнэ.
- Scheduler нь процессыг priority эсвэл round-robin зарчмаар сонгон ажиллуулах боломжтой байдлаар туршина.

Энэхүү лабораторийн ажил нь:

- Бодит kernel түвшний scheduler биш
- Бодит interrupt, timer interrupt биш
- Бодит CPU register save/restore хийхгүй

Харин дараах ойлголтуудыг сурахад чиглэнэ:

- Үйлдлийн систем процессуудыг **зөвхөн код** гэж үздэггүй, харин **удирдлагын мэдээлэлтэй объект** гэж авч үздэг
- CPU тухайн мөчид зөвхөн **нэг RUNNING процесс**-той байна
- Бусад процессууд **READY** эсвэл **WAITING** төлөвт байна
- CPU нэг процессоос нөгөөд шилжихэд **context switching** явагдана
- Зарим scheduler нь **priority**-г харгалзан процесс сонгодог

2. Онолын үндэс

2.1 Процесс гэж юу вэ?

Процесс гэдэг нь санах ойд ачаалагдсан, ажиллаж буй програм юм. Процесс нь зөвхөн код биш, мөн дараах мэдээллийг агуулна:

- Process ID
- Process State
- Program Counter
- CPU register-үүдийн утга
- Priority
- Санах ойн мэдээлэл
- I/O төлөв
- Бусад удирдлагын мэдээлэл

Эдгээрийг үйлдлийн систем ихэвчлэн **Process Control Block (PCB)** дотор хадгалдаг.

2.2 Process State

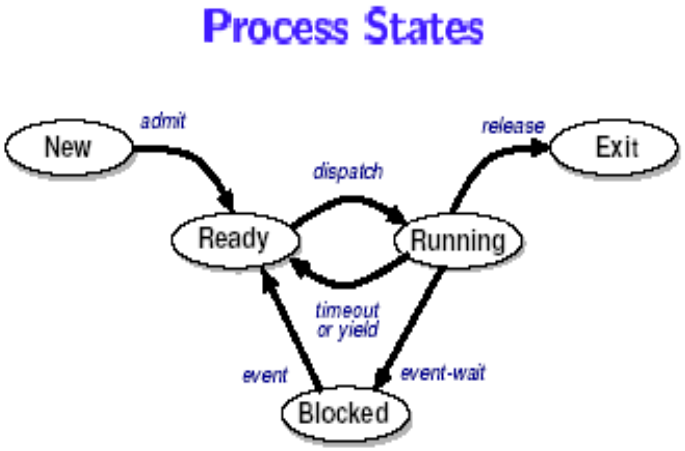
Процесс амьдралын мөчлөгийн явцад хэд хэдэн төлөвөөр шилжинэ.

Үндсэн төлөвүүд:

- **NEW** – шинээр үүссэн
- **READY** – CPU авахад бэлэн
- **RUNNING** – CPU дээр яг одоо ажиллаж байгаа
- **WAITING / BLOCKED** – ямар нэг үйл явдал, I/O, хугацаа хүлээж байгаа
- **TERMINATED / EXIT** – ажлаа дуусгасан

Төлөв шилжилтийн үндсэн жишээ:

- NEW -> READY
- READY -> RUNNING
- RUNNING -> WAITING
- WAITING -> READY
- RUNNING -> TERMINATED
- RUNNING -> READY *(time slice дуусах үед)*



2.3 Process Priority

Priority нь аль процесс илүү чухал болохыг илэрхийлнэ. Ихэнх системд:

- **өндөр priority** бүхий процесс CPU-г түрүүлж авна
- **бага priority** бүхий процесс хүлээх магадлалтай

Энэ лабораторид priority-ийг бүхэл тоон утгаар өгч болно.

Жишээ:

- 1 = хамгийн өндөр priority
- 2 = дунд priority
- 3 = бага priority

Эсвэл эсрэгээр нь ашиглаж болно. Гэхдээ аль дүрэм хэрэглэж байгаагаа програмдаа тодорхой тайлбарласан байх ёстой.

2.4 Context Switching

Context switching гэдэг нь CPU нэг процессын ажлыг түр зогсоож, өөр процессын ажлыг үргэлжлүүлэх үед явагддаг шилжилт юм.

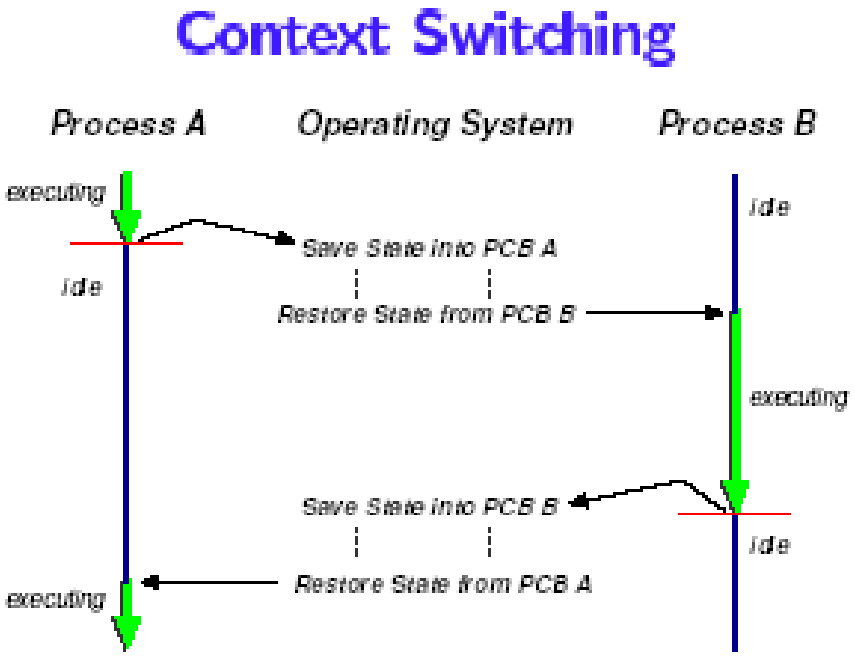
Бодит үйлдлийн системд:

- Одоогийн процессын context (register, program counter гэх мэт)-ийг хадгална
- Дараагийн процессын context-ийг сэргээнэ
- CPU шинэ процесст шилжинэ

Энэ лабораторийн симуляцид:

Бодит register хадгалахгүй, харин дараах байдлаар загварчилна:

- Одоогийн RUNNING процессын state = READY эсвэл WAITING болгоно
- Дараагийн процессын state = RUNNING болгоно
- process_counter, cpu_burst, waiting_time зэрэг утгуудыг шинэчилнэ
- “Context switching from Process X to Process Y” гэж хэвлэнэ



3. Ашиглах хэл, орчин

- **Хэл:** C
- **Компилятор:** gcc
- **Орчин:** Linux / Windows
- **Нэмэлт сангууд:**
  - stdio.h
  - stdlib.h
  - time.h
  - string.h
  - unistd.h *(Linux дээр sleep ашиглах бол)*

Windows орчинд Sleep() ашиглаж болно.

4. Ерөнхий шаардлага

1. struct PCB бүтцийг заавал тодорхойлно.
2. Дор хаяж **3 процесс** загварчилна.
3. Процесс бүр дараах мэдээлэлтэй байна:
   - process_id
   - priority
   - state
   - process_counter
4. Scheduler нь процесс сонгох логиктой байна.
5. Нэг мөчид зөвхөн **нэг процесс RUNNING** төлөвтэй байна.
6. Процессуудын төлөвийг динамикаар өөрчилнө.
7. Context switching явагдаж байгааг текстээр харуулна.
8. Програм олон алхамтай ажиллаж, процессуудын өөрчлөлтийг дэлгэцэнд хэвлэнэ.
9. rand() ашиглах бол srand(time(NULL));-ийг эхэнд нэг удаа дуудна.

10. Кодоо функцүүдэд хувааж, ойлгомжтой бичнэ.

5. PCB бүтэц

Доорх бүтэц бол санал болгож буй үндсэн хувилбар юм.

```
struct PCB {
   int process_id;        // Процессын дугаар
   int priority;          // Эрэмбэ / чухал чанар
   int state;             // Төлөв
   int process_counter;   // Хэдэн алхам ажилласны тоолуур
   int burst_time;        // CPU дээр ажиллах үлдсэн хугацаа
   int waiting_time;      // Хүлээлтийн хугацаа
};
```

6. Process State-ийн тогтмол утгууд

```
#define NEW 0
#define READY 1
#define RUNNING 2
#define WAITING 3
#define TERMINATED 4
```

Хэрэв хүсвэл enum ашиглаж болно:

```
typedef enum {
   NEW,
   READY,
   RUNNING,
   WAITING,
   TERMINATED
} ProcessState;
```

7. Туслах функцууд

Оюутнууд дараах туслах функцуудыг хэрэгжүүлэхийг зөвлөж байна.

7.1 state-ийг текст болгох функц

```
const char* getStateName(int state);
```

Жишээ гаралт:

- READY
- RUNNING

- WAITING
- TERMINATED

7.2 PCB мэдээлэл хэвлэх

void printPCB(struct PCB p);

Хэвлэх мэдээлэл:

- Process ID
- Priority
- State
- Process Counter
- Burst Time
- Waiting Time

7.3 Context switching хэвлэх

void contextSwitch(struct PCB *current, struct PCB *next);

Жишээ:

[Context Switch] Process 1 -> Process 2
Saving state of Process 1...
Loading state of Process 2...

8. Лабораторийн үндсэн даалгавар

Даалгавар 1 — 3 процесс үүсгэх

3 өөр процессыг PCB хэлбэрээр тодорхойлно.

Жишээ:

- **Process 1**
  - process_id = 1
  - priority = 1
- **Process 2**
  - process_id = 2
  - priority = 2
- **Process 3**
  - process_id = 3
  - priority = 3

Эхний төлөв:

- бүгд READY

Даалгавар 2 — Процесс бүрийн логик

Процесс бүр өөр “ажил” хийх ёстой. Жишээ нь:

Process 1

Random тоонууд үүсгээд эрэмбэлэх

Process 2

2 хэмжээст матриц нэмэх

Process 3

String доторх тэмдэгтийн давтамж тоолох

Эдгээр нь өмнөх лабораторийн логиктой холбоотой байж болно. Гэхдээ энэ удаад процессын **state**, **priority**, **switching**-тэй хамт ажиллах ёстой.

Даалгавар 3 — Scheduler хэрэгжүүлэх

Scheduler нь дараагийн ажиллах процессыг сонгоно.

Та дараах 2 аргын аль нэгийг сонгож болно.

Хувилбар A — Priority Scheduling

- READY төлөвтэй процессууд дундаас
- хамгийн өндөр priority-той процессыг сонгоно
- CPU дээр ажиллуулна

Жишээ дүрэм:

- priority утга **бага байх тусам өндөр эрхтэй**

Тэгвэл:

- Priority 1 > Priority 2 > Priority 3

Хувилбар B — Round Robin Scheduling

- Процессуудыг дараалаар нь ээлжлэн ажиллуулна

- Нэг удаад нэг алхам эсвэл нэг time quantum өгнө

Жишээ:

- P1 -> P2 -> P3 -> P1 -> P2 -> P3 ...

Даалгавар 4 — Process State шилжилт

Процессууд ажиллах явцдаа state өөрчлөгдөнө.

Жишээ логик:

- Scheduler сонговол: READY -> RUNNING
- Ажиллаж дуусаагүй ч ээлж нь дуусвал: RUNNING -> READY
- Түр хүлээх бол: RUNNING -> WAITING
- Хүлээлт дуусвал: WAITING -> READY
- Ажлаа дуусгавал: RUNNING -> TERMINATED

Даалгавар 5 — Context Switching симуляци

Хэрэв нэг RUNNING процессоос нөгөө процесс руу шилжвэл:

1. Одоогийн процессын төлөвийг шинэчилнэ
2. Дараагийн процессын төлөвийг RUNNING болгоно
3. Console дээр context switching мэдээлэл хэвлэнэ

Жишээ:

```
---------------------------------
CPU Scheduler selected Process 2
[Context Switch] Process 1 -> Process 2
Process 1 state: READY
Process 2 state: RUNNING
---------------------------------
```

Даалгавар 6 — WAITING төлөв оруулах

Симуляци илүү бодитой байлгахын тулд дор хаяж нэг нөхцөлд процесс WAITING төлөвт орно.

Жишээ:

- Process 2 матриц бодсоны дараа 2 алхам WAITING байж болно
- Эсвэл random нөхцөлөөр WAITING болгоно

Жишээ:

```
if (rand() % 4 == 0) {
   pcb->state = WAITING;
}
```

Дараа нь тодорхой алхмын дараа:

```
pcb->state = READY;
```

9. Програмын ажиллагааны загвар

Хувилбар A — Автомат scheduler simulation

Програм ажиллахад:

1. Бүх процесс READY төлөвтэй эхэлнэ
2. Scheduler дараагийн процесс сонгоно
3. Процесс 1 алхам ажиллана
4. State шинэчлэгдэнэ
5. Дараагийн процесс сонгогдоно
6. Энэ үйлдэл олон удаа давтагдана

Хувилбар B — Menu-based simulation

Цэс гаргаж өгнө:

1. Show all processes
2. Run scheduler one step
3. Run scheduler continuously
4. Change process priority
5. Exit

Энэ хувилбар нь сургалтын хувьд илүү ойлгомжтой байж болно.

10. Гаралтын жишээ

Initial Process Table

PID	Priority	State	Counter
1	1	READY	0
2	2	READY	0
3	3	READY	0

Scheduler selected Process 1
[Context Switch] CPU -> Process 1

Process 1 is RUNNING
Process 1 generated and sorted numbers
Process 1 counter = 1

Scheduler selected Process 2
[Context Switch] Process 1 -> Process 2
Process 1 state changed to READY
Process 2 is RUNNING
Process 2 added matrices
Process 2 counter = 1

Scheduler selected Process 3
[Context Switch] Process 2 -> Process 3
Process 3 is RUNNING
Process 3 counted character frequency
Process 3 counter = 1

11. Хэрэгжүүлэх шаардлагатай үндсэн функцууд

Оюутнууд дараах функцуудыг заавал эсвэл санал болгож хэрэгжүүлж болно:

```
void process1(struct PCB *p);
void process2(struct PCB *p);
void process3(struct PCB *p);

int selectNextProcess(struct PCB processes[], int n);
void contextSwitch(struct PCB *current, struct PCB *next);
void printPCB(struct PCB p);
void printAllProcesses(struct PCB processes[], int n);
```

12. Програмын логикийн зөвлөмж

Алхамчилсан хэрэгжилт

1-р үе шат

Зөвхөн PCB бүтэц зарлаж, 3 процесс үүсгэ

2-р үе шат

State болон priority-г хэвлэдэг болго

3-р үе шат

Scheduler функц бич

4-р үе шат

Процесс бүрийн логик ажиллуул

5-р үе шат

Context switching хэвлэдэг болго

6-р үе шат

WAITING болон TERMINATED төлөв нэм

13. Тайланд тусгах асуултууд

Оюутан тайландаа дараах асуултад хариулна:

1. PCB гэж юу вэ?
2. Process state ямар үүрэгтэй вэ?
3. READY ба RUNNING төлөвийн ялгаа юу вэ?
4. WAITING төлөв хэзээ үүсдэг вэ?
5. Priority scheduling ямар зарчмаар ажилладаг вэ?
6. Context switching гэж юу вэ?
7. Context switching яагаад overhead үүсгэдэг вэ?
8. Энэ лабораторийн симуляци бодит үйлдлийн системээс юугаараа ялгаатай вэ?

14. Хүргэлтийн бүтэц

Файлын бүтэц дараах байж болно:

- `Lab4.c`
- `README.txt`

эсвэл

- `pcb.h`
- `scheduler.c`
- `processes.c`
- `main.c`
- `README.txt`

---

15. Compile / Run жишээ

Linux

```
gcc Lab4.c -o lab4
./lab4
```

Олон файлтай бол

```
gcc main.c scheduler.c processes.c -o lab4
./lab4
```

---

16. Нэмэлт даалгавар

Сайн гүйцэтгэсэн оюутанд дараах өргөтгөлүүдийг хийж болно:

Нэмэлт 1

Priority-г runtime үед өөрчилдөг болгох

Нэмэлт 2

Aging механизм нэмэх
Удаан хүлээсэн процессын priority-г аажмаар өсгөх

Нэмэлт 3

Gantt chart-тэй төстэй текстэн гаралт хийх

Жишээ:

```
Time 1: P1
Time 2: P2
Time 3: P1
Time 4: P3
```

Нэмэлт 4

Process state transition diagram-ыг тайландаа зурж оруулах

---

17. Дүгнэлт

Энэхүү лабораторийн ажил нь оюутнуудад:

- PCB-ийн үүргийг ойлгох
- Процессын төлөвийн шилжилтийг ойлгох
- Priority-ийн нөлөөг харах
- Scheduler хэрхэн шийдвэр гаргадагийг мэдэх
- Context switching-ийн үндсэн санааг ойлгох

боломж олгоно.

Энэ нь дараагийн дараах сэдвүүдийг үзэхэд маш чухал суурь болно:

- CPU Scheduling Algorithms
- Preemptive Scheduling
- Multitasking
- Threads
- Synchronization
- Deadlock